

Interview quick reference — measured facts

Companion to `.claude/skills/lsm-integrity/references/interview-drills.md` . That file is *how to talk about it*. This file is *the numbers*, in tables, for fast recall.

Every number here is measured, not estimated. Sources: `serving/model_card.md` , `docs/stage6-scale-rehearsal.md` , `config/base.yaml` , and the source files named per section.

1. The project in 30 seconds

Magnetometer + GPS survey data over a buried pipeline → a risk-ranked list of places to dig. Seven stages: ingest → validate → features → detect → severity + classify → risk rank → growth. Dagster orchestration, MLflow tracking, SQLite + Parquet, Streamlit demo, CI gates.

Real LSM data is proprietary, so a physics-inspired synthetic generator stands in for it. The methodology is the deliverable, not the physical fidelity.

The headline: the detection model fails its own promotion gate. That failure is measured, reproduced at 800× scale, and reported rather than hidden.

2. The signal problem (why background removal *is* the project)

Quantity	Value
Ambient field (Bz)	~45,000 nT
Defect residual after detrend	25.05 nT
Defect signal as fraction of raw field	0.051%
Background residual after detrend	~7.8 nT
Detection contrast (defect / background)	3.02–3.21×
Sensor noise (per axis, per sample)	5.0 nT

Q: Does "detrend" just mean subtracting a polynomial? No — it's two steps, not one. Step 1: fit a smooth curve (a gentle degree-3 polynomial) to the slow background and subtract it. Step 2: slide a 40 m window along what's left and subtract the local median. Step 1 removes most of the ~45,000 nT background; step 2 mops up

whatever step 1 missed. The 25.05 nT and ~7.8 nT numbers above are measured *after both steps*. Full detail (including why median, not average) is in Section 6.

Q: Where's "interference noise" in this table? Is it 0.9 nT? It isn't in this table at all — there is no interference row here. The "Sensor noise" row (5.0 nT) is something else entirely: that's the electronic noise floor of the sensor itself, on every reading, defect or not. The 0.9 nT number lives in Section 4, and it means something narrower than "interference noise" — it's a **what-if**, not a real measurement: "0.9 nT is what an interference source *would* read if we didn't boost it." The corpus generator always applies that boost, so 0.9 nT never actually appears in the data. See the next question for why the boost exists.

3. Corpus geometry

	Demo corpus	Scale rehearsal
Lines	5	40
Length per line	2,000 m	40,000 m
Sample spacing	0.5 m	0.5 m
Runs per line	3	3
Surveys	15	120
Rows	60,000	9,600,000
Defects per line	12	240
Interference per line	4	80
Defect density	6 / km	6 / km (held constant)
Source depth	1.5 m	1.5 m
Gradiometer baseline	0.5 m	0.5 m

Q: What is "rehearsal" (the right-hand column)? It's the same generator, same settings, just run bigger: 40 lines instead of 5, each line 20× longer. Same code, same defect density (6 per km) — only the volume changes. The point is to check, before any real data exists, whether the pipeline and its conclusions still hold up when run at something closer to production size. "Demo corpus" = small and fast, used day-to-day. "Scale rehearsal" = the same thing at 800× the rows, used as a one-time stress test. Written up in `docs/stage6-scale-rehearsal.md`.

Q: Is 1/r³ actually used, or is that just a comment? It's real code, not just a comment. `dipole_field()` in `src/lsm/generate.py:57-65` computes the magnetic field from every defect and every interference source using the real physics formula for a magnetic dipole, and that formula divides by distance-cubed (`/ dist**3`),

line 65). This runs for every single defect and interference source when the corpus is generated — it's the actual field calculation, not a docstring.

Q: Why does interference get multiplied by $\times 50$, and what does that have to do with $1/r^3$? Because of that same $1/r^3$ formula: field strength drops off as the *cube* of distance. A defect sits 1.5 m from the sensor; interference sits roughly 5.5 m away. $(5.5 / 1.5)^3 \approx 50$ — so something 3.7 $\times$ farther away reads about 50 $\times$ weaker, purely from geometry, even before you account for its size. The generator counters this by giving interference sources a magnetic moment (their intrinsic "strength," independent of distance) about 50 $\times$ bigger than a defect's. Without that $\times 50$ boost, interference would read ~ 0.9 nT after the $1/r^3$ drop-off — below the noise floor, meaning it would never show up as a false positive. *With* the boost, interference ends up roughly as strong as a real defect (same amplitude) but visibly wider/blurrier (see Section 4's FWHM row) — which is the whole design: telling them apart has to come from *shape*, not strength.

Q: Why so many independent lines instead of packing defects onto one line? Two separate reasons, one practical, one statistical: - *Practical*: cramming 60 defects onto a single 2 km line made them sit close enough together that one defect's magnetic "tail" (which also follows that $1/r^3$ falloff) bled into a neighbouring defect's background reading, making the background noisier and hurting the detection-contrast gate (3.19 $\times$ down to ~ 2.7 – $3.0\times$). - *Statistical*: spreading the same defects across more independent lines also shrinks the measurement's confidence interval (see next question) — more independent samples means a more reliable average, at no extra cost.

Q: What does "CI" mean here, and why does it matter? This document uses "CI" for two *different* things — worth keeping straight: - **Statistical CI** ("confidence interval") — a range around a measured number showing how much it might shift if you reran the experiment on a different sample. E.g. "recall 0.639 [0.630, 0.648]" means: best estimate 0.639, but anywhere in that bracket is plausible too. This is what Sections 9–13 mean by CI, and what the $\sqrt{5}$ below is about. - **Software CI** ("continuous integration") — the GitHub Actions workflow (`ci.yml`) that runs tests on every code push. Unrelated to the statistics. Mentioned in Section 9.

Why the statistical CI matters: a single number on its own can be misleading. Saying "IsolationForest recall = 0.610" sounds like a fact, but if you ran the same experiment again on a slightly different set of defects, you might get 0.590 or 0.630 just from randomness — you don't know how much to trust 0.610 until you know its range. The CI gives you that range. This project's promotion gates go one step further and check the *pessimistic end* of that range (the lower bound), so a model only passes if it would still look good even in an unlucky sample — not just on its best-case average.

Where $\sqrt{5}$ comes from: the more independent samples you average, the narrower your confidence interval gets — specifically, width shrinks proportional to $1/\sqrt{\text{number of samples}}$. That's the same math behind "a bigger opinion poll has a smaller margin of error." So: take the same total number of defects and spread them across 5 $\times$ as many independent lines instead of 1 — the interval narrows by a factor of $\sqrt{5} \approx 2.24$. This is a general rule about a 5 $\times$ comparison, not the literal demo-to-scale jump (which is 5→40 lines, an 8 $\times$ increase) — that actual result is reported separately in Section 10.

4. Defect vs interference — the central design

	Defect	Interference
Lateral offset	0 m (under sensor)	3–8 m to the side
Distance to sensor	1.5 m	3.4–8.1 m
Moment	20–80	×50 larger
Grows across runs	yes, ×1.15 per run	no, static
Label half-width	3.0 m	6.7–16.3 m
Measured peak FWHM	2.2 m	6.2 m
Label channel	<code>defect=1</code> , <code>defect_type</code>	<code>interference=1</code> , <code>defect=0</code>

Q: Is "moment" the signal intensity? Not directly — moment is the *input* strength of the buried source (its magnetic dipole moment, an arbitrary unit here since the project doesn't use the real physics constant — see Section 15, weakness #2). What the sensor actually reads (the nT value) is moment combined with the $1/r^3$ distance drop-off and the source's orientation. So two sources with the same moment can produce very different readings if they're at different distances — moment is one input to the signal, not the signal itself. Code: `moment = unit(orientation) * severity * growth` , then `dipole_field()` turns that moment into an actual nT reading (`src/lsm/generate.py:193-195`).

Q: What is "grows across runs" — what's growing? A "run" is not a new line and not a new defect — it's the same physical line, scanned again later, as if the crew walked the same pipeline a second and third time. Per line, the generator picks each defect's and interference source's location and orientation **once** (`_build_features()`), then reuses that exact same set of features for every run (`run_id = 0, 1, 2` → 3 scans of the same line, matching Section 3's "Runs per line: 3"). The only thing that changes between those repeat scans is a defect's *severity* (its magnetic moment), which is multiplied by `1.15**run_id` — i.e. 1.0×, 1.15×, 1.3225× on runs 0, 1, 2 — modelling real corrosion or crack growth getting worse between visits. Interference severity has no such multiplier at all (always ×1.0), because a fence post or scrap metal doesn't change between scans the way corrosion does. `growth: 1.15` is a real config value in `config/base.yaml` , and it's also the exact number Stage 8's growth model is later asked to recover from the data (Section 9, Section 15 weakness #3). Code: `src/lsm/generate.py:191-195` (`growth = cfg.growth**run_id if is_defect else 1.0`), and note `_build_features` is called once per line while `_make_run` loops over `run_id` reusing its output — that's what guarantees "same defects, same spots, just later in time."

Q: Is "label half-width" a registered parameter? Yes, but it's a formula, not a single fixed number. `config/base.yaml` sets `label_window_scale: 2.0` . The actual half-width for any one source is `label_window_scale × r_eff` , where `r_eff` is that source's real straight-line distance to the sensor (`sqrt(depth_m2 + lateral_offset_m2)`). For a defect, lateral offset is 0, so `r_eff = depth_m = 1.5 m` , giving `2.0 × 1.5 = 3.0 m` — the table's defect value. For interference, lateral offset is 3–8 m, so `r_eff` ranges ~3.35–8.14 m, giving the table's 6.7–16.3 m range. This half-width defines the window of rows around a source's

location that get labelled `defect=1` / `interference=1` in training data — bigger `r_eff` (farther away) means a wider, blurrier footprint, which is part of why interference reads *broader* than a defect even at similar amplitude.

Why ×50: $(5.5/1.5)^3 \approx 50$, exactly the $1/r^3$ penalty for sitting 5.5 m off-axis. Without it, interference peaks at 0.9 nT — below the noise floor — and the false-positive trap traps nothing.

The thesis: *shape separates them, amplitude does not.*

Supporting EDA (PR-AUC):

Feature block	Defect vs background	Defect vs interference
Amplitude (~20 features)	up to 0.97	mostly < 0.55
<code>w25m_kurt</code>	—	0.985
<code>w25m_zcr</code>	—	0.786
<code>w10m_kurt</code>	—	0.680

Q: Why 20 "amplitude" features — we only have 3 magnetic axes and GPS? The 20 isn't per-sensor — it comes from one combined signal measured 5 ways at 4 window sizes. `features.py` first collapses the 3 axes (`rx`, `ry`, `rz`) into one magnitude series, `r_mag` (`sqrt(rx² + ry² + rz²)`, Section 6). Then, for each of the 4 rolling-window sizes (2 m, 5 m, 10 m, 25 m — Section 6's "Windows" block), it computes 7 statistics on that single `r_mag` series: mean, std, max, peak-to-peak, kurtosis, zero-crossing-rate, energy. Of those 7, **5 are about magnitude** (mean, std, max, peak-to-peak, energy) and 2 are about **shape** (kurtosis, zero-crossing-rate). "Amplitude (~20 features)" in this table means those 5 magnitude stats × 4 window sizes = 20 — deliberately excluding the kurtosis/ZCR columns, which is exactly *why* this row scores well against background (up to 0.97) but poorly against interference (mostly < 0.55): interference is designed to match a defect's amplitude, so amplitude-only features can't tell them apart (Section 4's whole thesis). GPS doesn't feed this table at all — it only supplies along-track distance (used to convert "25 m" into a number of samples via `step_m`), not a feature value itself. Code: `src/lsm/features.py:515-541`.

Examples of interference: fence post, buried scrap, well casing. **Not** an adjacent pipeline — that is a long line source, roughly constant along-track, which detrending removes rather than flags as a peak.

5. The 14 data-quality checks (`src/lsm/validate.py`)

Seven stop a survey, seven warn. Which is which is **configuration** (`config/base.yaml` → `validate.gates`), not code.

#	Check	Asks	Threshold	Action
1	<code>schema</code>	Columns, dtypes, nulls correct?	pandera schema	stop
2	<code>range</code>	Field value physically possible?	$\pm 80,000$ nT	stop
3	<code>saturation</code>	Axis repeating same value? (stuck ADC)	3 in a row	stop
4	<code>sample_idx_monotonic</code>	Sample numbers increasing?	—	stop
5	<code>duplicate_sample_idx</code>	Sample number repeated?	—	stop
6	<code>duplicate_content</code>	Same data already stored elsewhere?	content hash	stop
7	<code>survey_overlap</code>	Suspiciously identical to another run?	Pearson $r > 0.9$	stop
8	<code>sample_idx_gap</code>	Hole in the survey?	> 2.0 m	warn
9	<code>gps_jump</code>	Position jumped?	> 5.0 m	warn
10	<code>gps_chainage_consistency</code>	GPS path $\approx$ walked distance?	2% rel. error	warn
11	<code>noise_floor</code>	Sensor noise plausible?	1–15 nT	warn
12	<code>background_regime</code>	Field level unlike other runs?	$z > 3.0$	warn
13	<code>interference_density</code>	Too many strong outliers?	vs 5% expected	warn
14	<code>coverage</code>	Survey covers expected length?	95%	warn

How `schema` (#1) actually works: `pandera DataFrameSchema` objects in `schemas.py` — `RawReadingSchema` for raw rows, `FeatureFrameSchema` for the feature store — imported by every module that touches a `DataFrame` boundary (`generate.py` , `ingest.py` , `validate.py` , `features.py`). Both declared `coerce=False` : never repair the input, only reject it, because a boundary that silently coerces bad data is a boundary where training/serving skew hides. Failures raise `SchemaValidationError` carrying structured `failure_cases` (column, check, value, row index), not a bare string. *(Why pandera and not pydantic, given pydantic is already a dependency? Different jobs — pydantic validates one config object; pandera validates thousands of DataFrame rows at once, vectorised, which a pydantic model can't do without looping row-by-row.)*

How `duplicate_content` (#6) actually works: two independent hashes, not one, both in `hashing.py` . `file_sha256` is a streamed SHA-256 over the raw file's bytes — proves *this exact artifact*, but gives false negatives for dedup (re-export identical values with different float formatting and you get a different hash). `content_sha256` is a canonical hash: a fixed, declared column list in a fixed order, sorted by `sample_idx` , floats rounded to 6 decimals, then hashed over the Arrow buffers — that's what ingest actually keys on, and what this check compares against.

Caveat to volunteer if asked: `coverage` is written and tested but currently inactive — no caller passes `expected_length_m` , so it always returns pass with reason *"no expected length declared"*. 13 of 14 produce a real verdict today.

Quarantine, not crash. A hard failure records all 14 verdicts to `dq_report` , marks the survey `quarantined` , copies raw file + JSON report to a quarantine folder, and continues. Dagster yields `{"outcome": "skipped"}` rather than failing the run. *Rationale: a pipeline that halts a nightly run over one bad sensor gets switched off by its own operators.*

Measured at scale: 2 of 120 surveys (1.7%) quarantined by `survey_overlap` , both just above the fixed 0.9 threshold — a real, scale-driven finding (longer lines accumulate more correlated structure; the max is taken over a growing number of prior runs).

6. Features (`src/lsm/features.py`) — 46 columns

Detrend, two stages: robust degree-3 polynomial (IRLS, Tukey biweight, 4 iterations) → 40 m rolling-**median** high-pass. Median so a defect doesn't partly subtract itself. Window must stay >> defect footprint or the high-pass eats the signal.

Block	Columns	Notes
Residual	<code>rx/ry/rz_nt</code> , <code>r_mag_nt</code> , <code>r_incl_deg</code> , <code>r_decl_deg</code>	orientation is near-constant across a defect, random for noise
Along-track gradient	<code>dr_ds</code> , <code>d2r_ds2</code> , <code>drz_ds</code>	derivative of a single moving sensor — <i>not</i> gradiometry
Vertical gradiometry	<code>gx/gy/gz_nt_per_m</code> , <code>g_mag</code>	true two-head common-mode rejection
Windows (2/5/10/25 m)	mean, std, max, ptp, kurt, zcr, energy	7 stats × 4 windows = 28
Peak shape	<code>fwhm_m</code> , <code>peak_asymmetry</code> , <code>decay_exponent</code> , <code>peak_prominence_nt</code> , <code>peak_distance_m</code>	the real discriminators

Two details worth quoting: - `zcr` is computed on `rz` (**signed**), **not** `|r|` — a magnitude never crosses zero, so `|r|`-based ZCR would be identically zero. A feature that looks computed and isn't. - `decay_exponent` **did not work as intended:** -0.95 on-pipe vs -0.84 off-pipe, no useful separation. Kept because it measures how dipole-like an anomaly is; negative result documented beside the feature.

Leakage prevented by type signature, not convention. `StatelessTransform` is a callable of exactly `(survey_df, SurveyContext) -> DataFrame` , where `SurveyContext` carries only `survey_id`, `line_id`, `run_id`, `step_m`, `standoff_m`, `surveyed_at`, `gradiometer_baseline_m` — no corpus, no DB connection. A function with that signature *cannot* reach across surveys even if someone tried; it's legitimately re-fit at inference because it's background removal, not learned state. `FittedTransform` is a separate ABC with `fit()` / `transform()` guards that raise `NotFittedError` / `AlreadyFittedError` — fitting twice is the same training/serving skew this project is paranoid about everywhere else.

6b. Plain-language walkthrough

The problem this section solves: the raw sensor reading is mostly Earth's background magnetic field, which is huge (~45,000 nT) and changes slowly as you walk. The actual defect signal is tiny by comparison. So step one is to strip away the slow background and keep only the small leftover bumps. Step two is to describe the *shape* of those bumps with numbers, so a model can later tell "this bump is a real defect" from "this bump is a fence post."

Step 1 — removing the background (detrending), done in two passes: 1. Fit a smooth curve through the data (a gentle degree-3 polynomial) to approximate the slow background drift, and subtract it off. This fitting uses a trick (Tukey biweight, 4 rounds) that automatically down-weights the actual defect bumps while fitting, so a defect doesn't drag the curve toward itself. 2. Slide a 40-meter window along the data and subtract the local median value. This mops up any slower wiggle the polynomial missed.

Why *median* and not *average*: an average would partly absorb the defect's own bump into the "background" estimate and cancel some of it out. A median mostly ignores a small number of unusually high/low points (like the defect), so the defect survives.

Why 40 meters specifically: the window has to be much wider than a defect's own footprint (defects are only ~2–3 m wide). If the window were too close to the defect's size, this smoothing step would start eating the defect signal itself along with the background.

What's left after both subtractions is called the "residual" — literally just "whatever's left over." The columns `rx_nt`, `ry_nt`, `rz_nt` are that leftover value in each of the 3 sensor axes, `r_mag_nt` is their combined strength, and `r_incl_deg` / `r_decl_deg` describe which direction the leftover field is pointing. Near a real defect, that direction stays roughly consistent as you pass over it (because it's coming from one solid object). Near plain sensor noise, the direction is random from one reading to the next.

Step 2 — measuring how fast things change ("gradient" features):

This is not solving any physics equation — it's just subtraction and division, i.e. "how much did the reading change, over what distance." Two different versions:

- *Along-track gradient* (`dr_ds`, `d2r_ds2`, `drz_ds`): take two back-to-back readings from the same single sensor as it walks forward, subtract them, divide by the distance. That's a plain slope/derivative. Uses only one sensor.
- *Vertical gradiometry* (`gx/gy/gz_nt_per_m`, `g_mag`): the synthetic sensor rig has two magnetometers stacked 0.5 m apart on a pole. Subtract the bottom reading from the top reading, divide by 0.5 m. Because both sensors are close together, they both see almost the same distant background field — subtracting cancels that shared background out (like noise-cancelling headphones), ideally leaving mostly the nearby defect's signal. This one genuinely uses two physical sensors at once, not math on a single sensor's history.

(Separately documented as a negative result in Section 7: the two-sensor subtraction actually made detection *worse* here, because subtracting two noisy sensors adds up their noise faster than it removes background that a

plain detrend had already mostly removed.)

Step 3 — rolling statistics ("windows"):

For 4 different window sizes (2 m, 5 m, 10 m, 25 m), slide that window along the residual and compute 7 simple stats each time: mean, standard deviation, max, peak-to-peak range, kurtosis (how "spiky" the distribution is), zero-crossing rate (how often the signal flips sign), and energy (roughly, total signal strength). That's 7 stats × 4 window sizes = 28 columns. Same idea as a moving average, just computing more than one summary number at each position.

Step 4 — describing the shape of a detected bump ("peak shape"):

Once a candidate bump is found, measure things like how wide it is at half its height (`fwhm_m`), whether it's lopsided (`peak_asymmetry`), how sharply it decays outward (`decay_exponent`), how tall it stands above the surrounding noise (`peak_prominence_nt`), and how far it is from the nearest other peak (`peak_distance_m`). These shape measurements are "the real discriminators" — the features that actually separate a real pipe defect from random nearby junk, since (per Section 4) amplitude alone can't tell them apart, only shape can.

Two specific gotchas worth knowing: - `zcr` is computed on the *signed* vertical residual (`rz`), not on the combined magnitude (`r_mag`). Deliberate: a magnitude value is always positive, so it can never cross zero, which would make a magnitude-based zcr always equal to 0 — a feature that looks like it's measuring something but isn't. - `decay_exponent` was meant to measure how "point-source-like" a bump's falloff is, but it didn't actually separate real defects from interference in testing (−0.95 vs −0.84, too close to tell apart). Kept in the feature set anyway, with that negative result written down next to it, rather than quietly dropped.

One design guarantee worth knowing: every one of these calculations only ever looks at data from *one single survey run* at a time — it has no way to see or reach into any other survey in the database while running. That's enforced by the function signature itself (it's only ever handed one survey's data plus a few metadata numbers, nothing else), not just by a coding convention someone could forget. That matters because it's what guarantees a feature computed at prediction time can't accidentally "peek" at information it wouldn't have in the real world — a common way ML pipelines quietly cheat during testing without anyone noticing.

7. The gradiometer — a negative result

Configuration	Detection contrast
Single head + detrend	3.21×
Vertical gradiometer	1.45×

The difference operation cancels the background but accumulates both sensors' noise ($\sigma\sqrt{2}$). After detrending already removed most of the background, you pay the noise cost for a benefit you already had. Gradiometry wins when the background is strongly non-uniform along-track — not here.

The self-deception that had to be fixed first: the second head was originally built from the *same* drift/wave arrays as the first, so the background's vertical gradient was exactly zero and common-mode rejection was perfect **by construction**. Head-difference std was 6.91–7.02 nT = $\sigma\sqrt{2}$ exactly, i.e. pure sensor noise. After modelling the real main-field gradient (0.02 nT/m) and geology separately: 6.98–8.37 nT, i.e. ~2.4 nT of genuine background gradient.

7b. Plain-language walkthrough

What "detection contrast" means: how many times taller a real defect's bump stands above the leftover background noise, after processing. Bigger number = defect is easier to spot. 3.21× means the defect signal is about 3.21 times the size of typical background wobble.

What was tried: instead of one magnetometer, use two, stacked 0.5 m apart on a pole (see Section 6's "vertical gradiometry"). Subtract the bottom reading from the top. The hope: since both sensors see almost the same distant background, subtracting cancels the background out and leaves a cleaner defect signal — like noise-cancelling headphones.

What actually happened: it made things worse, dropping contrast from 3.21× down to 1.45×. Two reasons combine: 1. The background had *already* been mostly removed by the detrending step from Section 6. So there wasn't much background left for the two-sensor subtraction to usefully cancel. 2. Subtracting two independent sensors doesn't just cancel their shared background — it also *adds together* their independent random noise. Two noisy measurements combined have about 1.4× ($\sqrt{2}$) more noise than either one alone. So you pay a real noise cost for a background- cancelling benefit that was mostly already gone.

Rule of thumb this taught: the two-sensor trick is worth it when the background itself changes a lot as you move along the pipe (so a plain detrend can't remove it, but two sensors can). In this data, the background is fairly smooth along the pipe, so a plain detrend already did the job — the two-sensor subtraction had nothing left to win, only noise left to add.

The bug that had to be caught first ("the self-deception"): early in building the simulator, the fake "top" and "bottom" sensors were accidentally generated from the exact same underlying background data. That means, by construction, there was *zero* difference between what the two sensors saw — so subtracting them looked like a perfect result, but only because the test was rigged, not because the technique actually worked. It's the simulation equivalent of grading your own exam with the answer key already filled in.

This was caught by checking the numbers: the leftover noise after subtracting was *exactly* what you'd expect from pure sensor noise alone ($\sigma\sqrt{2}$, i.e. no real background signal left at all) — a suspiciously clean result. The fix was to make the simulator model a small but real vertical difference in the background field (Earth's own field is not perfectly uniform with height, plus local geology adds more unevenness) instead of copying the same fake data to both sensors. After that fix, the two-head difference reflected real numbers again: mostly sensor noise, plus a small genuine ~2.4 nT background gradient — a realistic result instead of a rigged one, and the basis for the "vertical gradiometry didn't help here" finding reported above.

8. The observatory reality check

The synthetic background is a linear drift + one sinusoid — i.e. **polynomial-like by construction**, so a polynomial detrend flatters itself. Verified against real data.

Source: USGS Geomagnetism Program, station BOU (Boulder), public domain. Two days: quiet (2024-03-15) and the G5 "Mother's Day" storm (2024-05-10).

Background	Bare degree-5 poly residual	Two-stage detrend residual	Contrast
Synthetic	5.03 nT (= noise floor exactly)	7.81 nT	3.21×
Real, quiet	—	7.83 nT	~3.0×
Real, storm	6.9–11.3 nT	8.09 nT	3.02×

Conclusion: a bare polynomial does *not* fit real geomagnetic structure the way it fit mine; the actual two-stage method survives. Off by default — the storm residual would break the Stage 2 detectability gate, so it's a stress test, not the operating point.

9. The four models and their gates

Four separate models sit downstream of feature extraction, each answering a different question about an indication, and each held to its own promotion gate — a numeric bar it must clear before it's allowed to be called "working."

Stage	Model	Baseline	Gate	Measured on
3 Detect	IsolationForest (46 feat, contamination 0.03, 300 trees)	MAD robust z-score	recall gap $\geq$ 0.15 at CI lower bound	per-defect recall @ dig budget
4 Severity	LightGBM quantile (.05/.5/.95) + split conformal	global mean	coverage in [0.87, 0.93] , <i>point estimate</i>	per-defect
5 Classify	LightGBM multiclass + isotonic	majority class	SCC recall $\geq$ 0.90 at CI lower AND SHAP denylist	per source
8 Growth	Empirical-Bayes pooled log-linear	no-growth	beat baseline MAE at CI lower bound	per defect

Stage 3 — Detect: "is this even a defect?" IsolationForest scores every row for how anomalous it looks, and the pipeline compares its recall (how many real defects it actually finds, within a fixed dig budget) against a simple robust z-score threshold (MAD — median absolute deviation) as the baseline. The gate demands IsolationForest beat MAD by at least 0.15 recall, and that gap has to hold even at the *pessimistic* end of the confidence interval (the CI lower bound), not just on average. See Section 11 for why it currently fails this.

Stage 4 — Severity: "how bad is this defect?" Severity here is measured in **%SMYS** — percent of the pipe's Specified Minimum Yield Strength, i.e. how close the defect pushes the pipe toward its structural failure point (100% SMYS is the theoretical limit; higher is worse — see the vocabulary table in Section 16). This is a genuinely continuous number, not a category, and because it feeds real engineering decisions about which sites to dig up, a single point estimate isn't enough — the pipeline needs to know *how confident* that estimate is. That's why the model here is a **LightGBM quantile regressor**: instead of predicting one severity number, it's trained with three different quantile loss functions to predict the 5th percentile, the 50th percentile (median), and the 95th percentile of severity. Together those three numbers form a range — "probably between X and Y, most likely around Z" — rather than a single guess. On top of that, **split conformal** calibration is applied: a statistical correction, computed on held-out data, that adjusts the model's raw interval so it actually contains the true value the stated fraction of the time, rather than trusting the model's own (often overconfident) uncertainty estimate. The gate then checks that this final calibrated interval contains the true severity value between 87% and 93% of the time (target ~90%) — not too narrow (overconfident, misses the truth too often) and not too wide (technically safe but useless for decision-making). The baseline here is simply "always predict the global mean severity," to confirm the model is doing better than knowing nothing about the specific defect.

Stage 5 — Classify: "what kind of thing is this?" A different kind of question entirely — not "how severe" but "which category": corrosion, dent, weld anomaly, SCC (stress corrosion cracking), or interference (i.e. not a real pipe defect at all, see Section 4). This determines what physical action makes sense and how much the finding should weigh in risk ranking (Section 15 lists the per-type consequence weights: SCC 1.0, corrosion 0.6, dent 0.5, weld 0.4, interference 0.0). Because this is a *category* label rather than a number, the model is a **LightGBM multiclass classifier** — for each indication it outputs a probability per candidate class (a softmax-style objective) rather than a range around a single number. Raw probabilities out of a tree-based classifier tend to be poorly calibrated (e.g. it says "80% confident" but is only actually right 60% of the time in that bucket), so **isotonic regression** is applied afterward to correct those probabilities against held-out data — the classification equivalent of what split conformal does for the severity interval. The gate requires recall on SCC specifically (the most safety-critical category) to be at least 0.90 at the CI lower bound, since missing a real SCC finding is the worst kind of error this stage can make. The baseline is simply "always guess the majority class."

Why two separate LightGBM models rather than one that does both: severity and defect-type are different kinds of prediction problem — one is regression (a number on a continuum, wanting an honest uncertainty range), the other is classification (a label from a fixed set of categories, wanting calibrated confidence per class). LightGBM supports both objectives, but the loss function, the calibration method, and the promotion gate that makes sense for each are all different, so they're trained, evaluated, and gated as two independent models rather than one model doing double duty.

Stage 8 — Growth: "how fast is this defect getting worse across repeat surveys?" An Empirical-Bayes pooled log-linear model, checked against a "no growth" baseline (i.e. assume a defect's severity doesn't change between surveys). Its gate is the simplest of the four: just beat that baseline on mean absolute error, at the CI lower bound.

Note the asymmetry in how gates are checked: three of the four gates (Detect, Classify, Growth) use the CI *lower bound* — the pessimistic end of the confidence interval — because they're one-sided questions ("is it at

least this good?"). Severity coverage uses the *point estimate* instead, because it's inherently two-sided: an interval that's simply very wide would also clear a lower-bound check trivially (wide enough to always contain the truth) while being useless in practice, so that check has to land close to the target rather than merely clear a floor.

Physics-consistency gate (Stage 5, additional to the recall gate): no feature that encodes *where along the pipe* an indication sits (`chainage_m` , `sample_idx` , `chainage_peak/start/end_m`) is allowed to appear in the classifier's top 10 most-influential features (measured by TreeSHAP). If position predicts defect type, the most likely explanation isn't physics — it's that the model memorised which specific spots in the training corpus happened to have which label, i.e. leakage. Currently **PASSES**: the top features are shape-based (`w25m_kurt` , `r_dec1_deg` , other window statistics), not positional.

Exit code: `lsm train` only exits non-zero on the **Stage 3 (Detect) gate**. Stages 4 and 5 are computed, printed, and logged to MLflow, but their failure doesn't fail the command — there may legitimately be too few matched indications in a given run to evaluate them meaningfully, so treating their failure as fatal would make the tool unreliable for reasons unrelated to model quality.

Why this is two separate GitHub Actions workflows, not one: `ci.yml` (ruff, mypy, `pip-audit` informational-only, the full pytest suite, a CLI smoke run that stops short of actually training) runs on every push and is expected to always stay green — it's checking that the *code* works. `train.yml` is triggered manually or by a `train-*` tag, actually runs `lsm train` , and its exit code is the Stage 3 gate above — so it's expected to be red until a candidate model actually clears the bar; it's checking whether the *model* works. Splitting them means a real model failure never gets mistaken for a broken build, and a real code bug never gets to hide behind "the model's just not good enough yet."

10. Results — demo vs scale

Detection (the gate that fails)

Metric	Demo (60 defects)	Scale (9,600 defects)
MAD recall @ budget	0.644	0.639 [0.630, 0.648]
IsolationForest recall	0.639	0.610 [0.600, 0.618]
Recall gap (IF – MAD)	–0.006 [–0.067, +0.050]	–0.029 [–0.033, –0.026]
Gate (≥ 0.15 at CI lower)	FAIL	FAIL

The demo interval **contains zero** — statistically inconclusive. The scale interval is narrow and excludes zero. Scaling 800× resolved the *uncertainty* without changing the *sign*. **"Not enough data" is refuted.**

Where IsolationForest actually loses (scale)

Percent of all dig-budget holes	MAD	IsolationForest
Found no defect	23.3	26.6
— landed on interference	23.3	23.0
— landed on empty ground	0.0	3.6
Localisation error (m)	0.284	0.423
PR-AUC (row-level, diagnostic)	0.332	0.383

Every one of MAD's false digs lands on real interference. IsolationForest wastes 3.6% of the budget on nothing at all and localises ~50% worse. Its interference advantage (0.3 points) is **ten times smaller** than its recall loss (2.9 points).

Note IF has the **higher row-level PR-AUC** — it ranks rows better, then loses that advantage in the row → cluster → top-N aggregation.

The other three gates

Stage	Demo	Scale	Gate
Severity coverage (nominal 0.90)	0.920 [0.867, 0.967]	0.896 [0.891, 0.901]	PASS
Severity MAE (vs baseline)	7.46 (vs 15.09)	3.36 (vs 15.02)	—
SCC recall	0.125 [0.000, 0.264]	0.636 [0.620, 0.652]	FAIL (need 0.90)
Interference precision	0.909	0.999	—
Growth population rate	0.1398 (truth: ln 1.15 = 0.1398)	—	PASS

11. Why the detection model fails — the two-level answer

Level 1 — IsolationForest is a noisy re-implementation of MAD. It picks split features uniformly at random. ~20 of 46 features are amplitude features, all correlated with `r_mag_nt`, which is exactly what MAD thresholds. So most splits do MAD's job with extra variance. Evidence: worse localisation, digs on empty ground, same recall.

Level 2 — the task was framed wrong. Interference was designed to have the *same amplitude* as a defect, so it is exactly as statistically unusual as a defect. An anomaly detector finds unusual things; being unusual is what the two **share**. Separating them requires knowing that narrow = damage and wide = junk, which is supervised information.

Proof the data is fine: the supervised classifier reaches **0.999 interference precision** on the same features. The information is present; the unsupervised model can't use it.

Also worth separating: "fails the gate" and "worse than baseline" are two different claims. The gate isn't "beat the baseline" — it's "beat the baseline by at least 0.15 recall, and be confident about that even in the worst case." Concretely: at scale, MAD recall is 0.639. For IsolationForest's recall *gap* over MAD to clear +0.15, its own recall would need to land around 0.789 or higher. And because the gate checks the **CI lower bound** of that gap, not its average, 0.789 is actually the *floor* of what's needed — IsolationForest would need a real point estimate somewhat above that, so that even the pessimistic edge of its own confidence interval still clears +0.15. Landing at exactly 0.789 on average, with any statistical uncertainty at all, would still fail.

That threshold is deliberately steep, which is what the next point is about: imagine a different, *genuinely* improved model that pushed recall from 0.639 to 0.689 — a real, honest +0.05 gain, not noise. That model would still fail this gate, because +0.05 falls well short of the required +0.15. Failing the gate does not automatically mean "this model is bad" or "no better than guessing" — it can just mean "not enough of an improvement to justify swapping out a simple, well-understood baseline for a more complex, harder-to-explain model."

"Both happen to be true here" points at two separate facts about IsolationForest that didn't have to coincide: 1. It fails the strict +0.15 gate — true of almost any model, including a modestly better one. 2. It isn't even a modest improvement — its measured recall (0.610) is actually *lower* than MAD's (0.639), a **negative** gap (−0.029 at scale), not just an insufficiently small positive one.

Fact 2 is the stronger, more damning result, and it's logically independent of fact 1. A model could fail the gate while still being a real (if insufficient) improvement over the baseline — that would be a "good problem": a promising candidate worth iterating on further. Instead, IsolationForest fails on both counts at once: not only does it miss the bar this project set, it doesn't even clear the much lower bar of "better than the naive baseline at all." Keeping these two claims distinct matters for how the result gets reported — it separates "our promotion bar is strict" (a design choice) from "the model actually underperforms" (a finding about the model) — and here, both statements happen to be true simultaneously.

12. Three real bugs found only at scale

Bug	Mechanism	Impact
NaN severity poisoning	36 of 18,917 matched indications (0.4%) had NaN <code>severity_smys</code> ; one NaN into <code>np.quantile</code> NaN'd the whole fold's conformal margin	0.4% data issue → 0% coverage across the entire result
Dead config	<code>classify.n_estimators / num_leaves</code> never threaded into the model; silently matched hardcoded defaults	SCC recall 0.50 → 0.64 after fix
Small-data settings don't scale	<code>num_leaves: 7</code> (needed at 60 defects) caps quantile trees regardless of data volume	conformal coverage → 0% at 9,600 defects

13. Evaluation discipline (the part that signals experience)

Practice	Detail	What it means / what it outputs
Grouping	by <code>(line_id, 100 m block)</code> , never block alone; fold = hash of group key, so growing the archive never reshuffles existing folds	How train/test splits are drawn. Every row from the same 100 m stretch of the same line goes into the <i>same</i> fold, never split across train and test — otherwise a defect's neighbouring rows could leak into both sides and inflate the score. Output: a fixed assignment of each block to a fold, stable even as more surveys get added later.
Bootstrap	1,000 resamples, 95%, resampling unit differs per metric	The general technique used to turn one point estimate into a confidence interval: resample the evaluation set (with replacement) 1,000 times, recompute the metric each time, and read the 95% interval off that spread of 1,000 values. Output: not a single number but a range (e.g. "recall 0.639, CI [0.630, 0.648]") — the width tells you how much to trust the point estimate.
— recall	per physical defect (3 runs re-observe the same 12 defects = 12 samples, not 36)	The <i>unit being resampled</i> for the recall metric is one real-world defect, not one row or one survey observation. Since each defect gets surveyed 3 times, resampling by row would triple-count it and make the interval falsely narrow (falsely confident). Output: recall's CI reflects uncertainty over ~12 independent defects, not 36 correlated observations of them.
— false-dig rate	per survey	The unit being resampled here is one whole survey run — how many of <i>that survey's</i> dig recommendations turned out to be wrong. Output: a rate per survey, bootstrapped across surveys, so one especially bad survey can't be split up and over-counted.
— localisation	per matched dig	The unit being resampled is one successfully matched dig (a predicted location that got paired to a real defect). Output: typical distance error in metres between predicted and true defect location, with a CI over the population of matched digs.
— class recall	per physical source	The unit being resampled is one real physical defect source (e.g. one corrosion patch), not one row of sensor data about it. Output: what fraction of real sources of a given type got correctly classified, with uncertainty over the count of distinct sources, not rows.
Model comparison	paired bootstrap (same indices both models) — correlated per-defect rates mean two separate CIs can overlap while every paired resample favours one model	When comparing two models (e.g. MAD vs IsolationForest), the <i>same</i> resampled indices are reused for both models in each of the 1,000 rounds, rather than resampling each model independently. Output: for every resample, which model won on that resample — letting you say "model A beat model B in 950 of 1,000 resamples" even in cases where their two separate, independently-computed CIs would visually overlap and look inconclusive.
ROC-AUC	banned outright — positives are ~2.7% of rows	Receiver-Operating-Characteristic AUC would normally summarise how well a score ranks positives above negatives, from 0.5 (random) to 1.0 (perfect). It's disallowed here because with positives at only ~2.7% of rows, the metric is dominated by how well the model handles the easy 97.3% of true negatives

Practice	Detail	What it means / what it outputs
		— it can look deceptively high while still being useless at actually finding defects.
PR-AUC	computed, explicitly labelled diagnostic; the gate is indication-level	Precision-Recall AUC: plot precision (of the positives you flagged, how many were real) against recall (of the real positives, how many you caught) at every possible score threshold, then take the area under that curve — a single 0–1 number. Unlike ROC-AUC it never gives credit for true negatives, so it stays meaningful when positives are rare. It's reported here only as a <i>diagnostic</i> on raw row-level scores — it does not decide pass/fail; the actual promotion gate is measured after rows are clustered into indications (dig decisions), which is a different, coarser unit than the row-level score PR-AUC summarises.
Matching	greedy closest-first, tolerance 15 m, no truth source claimed twice	How a predicted indication gets paired to a real, known defect for scoring: take the closest unclaimed prediction-truth pair within 15 m, match it, remove both from the pool, repeat. Output: a one-to-one pairing where a single lucky prediction can't be double-counted against the same real defect twice.
Point-in-time	as-of join filters <code>surveyed_at > as_of</code> ; a separate assertion re-checks on the way out	Guards against evaluating a model using data it couldn't actually have had yet: when scoring a decision made at a given point in time, only surveys recorded <i>after</i> that point are excluded from what the model is allowed to see. Output: an evaluation that reflects what the model would genuinely have known in production, not one quietly boosted by future data; a second assertion re-verifies no future rows slipped through after the join.

The same discipline applied to code, not just statistics: 276 tests across ~20 files, each named for the property it guarantees rather than the module it covers — `test_leakage.py` (no group crosses a CV fold), `test_bundle_roundtrip.py` (save→load reproduces identical predictions), `test_golden.py` (a fixed tiny survey's output never silently changes), `test_dagster_backfill.py` (a killed-mid-run backfill doesn't reprocess). `test_features.py` uses `hypothesis` for property-based tests — the one module worth throwing random inputs at, since detrend/gradient math has to hold for values a hand-picked example set wouldn't think to try. `mypy src/lsm` type-checks the pipeline (not the app layer), with `ignore_missing_imports` set once in `pyproject.toml` so CI and every local run behave identically. The Streamlit app itself is tested via `streamlit.testing.v1.AppTest.from_file(...).run()` — the real script through Streamlit's actual runtime, not a hand-rolled approximation of it.

14. MLOps mechanics

Concept	Implementation
Raw immutability	<code>register_survey()</code> keyed on <code>(line_id, run_id)</code> : no row → insert; identical <code>content_sha256</code> → no-op; changed hash → hard <code>IngestConflictError</code> . No <code>UNIQUE(content_sha256)</code> constraint on purpose — an exact duplicate under a different key is left for the <code>duplicate_content</code> DQ check to quarantine gracefully, not an uncaught <code>IntegrityError</code>
Two hashes	<code>file_sha256</code> (raw bytes, provenance) vs <code>content_sha256</code> (canonicalised: fixed column order, sorted, floats rounded to 6 dp, Arrow buffers)
Corpus hash	<code>data_sha256</code> = Merkle-style hash over sorted member content hashes
Feature cache key	<code>content_sha256</code> + <code>feature_version</code> — content alone misses the skew from changing <code>features.py</code>
<code>feature_version</code>	is a directory : <code>features/fv=2/line_id=.../run_id=...</code> — two versions coexist on disk
Deployable unit	<code>pipeline_release</code> row = up to 4 models + <code>feature_version</code> + <code>schema_version</code> , promoted/rolled back together
Version string	<code>{date}-{short_sha}</code> , e.g. <code>2026.07.31-79453eae-dirty</code>
Bundle guards	hard-fail on mismatched <code>feature_version</code> , <code>schema_version</code> , or numpy / scikit-learn / lightgbm version
Provenance triple	<code>config_sha256</code> + <code>git_sha</code> (with <code>-dirty</code>) + <code>data_sha256</code> , in bundle, DB row, and MLflow
Promotion status	every release is <code>alias: "challenger"</code> — nothing has ever been promoted
Dig budget	5 per km → 10 holes for a 2 km survey
Config layers	<code>pydantic BaseModel</code> , two independent layers: code config (<code>config/base.yaml</code> — hyperparameters, feature specs, gates) is the <i>only</i> thing hashed into <code>config_sha256</code> ; environment config (<code>dev.yaml</code> / <code>prod.yaml</code> — paths, DB URI, concurrency) is never hashed, so identical code+data can't hash differently between dev and prod
Dependency pinning	<code>numpy==2.5.1</code> , <code>scikit-learn==1.9.0</code> , <code>lightgbm==4.6.0</code> — exact-pinned, not a floating range, so a fresh <code>pip install -e .</code> reproduces what the committed bundles were actually trained against; <code>bundle.py</code> hard-fails otherwise
Serialization	<code>joblib.dump</code> , one <code>bundle.joblib</code> per model per task; library versions are stamped by <code>save_bundle</code> itself at save time (what was actually importable), not supplied by the caller
Orchestration	Dagster, 3-asset graph (<code>registered_survey</code> → <code>dq_report</code> → <code>survey_features</code>), partitioned per-survey; asset <code>code_version</code> = <code>feature_version</code> , so bumping the feature code marks exactly the affected partitions stale — the specific reason Dagster was chosen over Airflow
Retry policy	deliberately asymmetric: 3× exponential backoff on <code>registered_survey</code> (genuine I/O), zero retries on <code>dq_report</code> / <code>survey_features</code> — a DQ failure is never an exception (<code>validate_raw_survey</code> returns a report, it doesn't raise), so "no retry on DQ failure" holds by construction

Concept	Implementation
SQLite concurrency	<code>PRAGMA journal_mode=WAL + foreign_keys=ON ; check_same_thread=False</code> only for the Streamlit session (its reruns land on different worker threads); writes serialised at the application level, one connection per writer
Storage split	SQLite = small relational control plane (registry, DQ, lineage, indications); Parquet = bulk append-only signal/features. Measured at 9.6M rows: SQLite's own insert path, not the Python row-conversion step, is the majority cost — it stops being the right tool at concurrent multi-writer ingest, not at this data volume

Database schema — the tables that matter:

Table	Primary key	Purpose
survey	survey_id , UNIQUE(line_id, run_id)	registry + hashes + status; deliberately no UNIQUE(content_sha256)
reading	(survey_id, sample_idx) , WITHOUT ROWID	the one bulk table in SQLite
dq_report	append-only	all 14 verdicts, every validation pass
model_run	model_version	one row per trained model (incl. non-releasable baselines)
pipeline_release	pipeline_version	the deployable unit — FKs to up to 4 model_run rows + feature_version + schema_version
indication	indication_id	FKs to survey and pipeline_release

The FK from pipeline_release to model_run is enforced (`PRAGMA foreign_keys=ON`), which is why a Streamlit demo session seeds model_run rows *before* pipeline_release rows when bootstrapping from the baked manifest — insert order isn't arbitrary.

Known inconsistency to be ready for: `train.py:1189-1195` hardcodes the IsolationForest as the release's anomaly_version . MAD is logged as a model_run but marked "n/a (baseline, no artifact)" and never releasable. So the demo serves the model that measured worse. Nothing was promoted, so it isn't strictly wrong — but there's no fallback to the better baseline.

15. Weaknesses to volunteer before they're found

Interviewers trust candidates who surface their own gaps.

1. **The detection model loses to a robust z-score threshold.** Measured, reproduced at 800× scale, reported in the model card and a red CI badge.

2. **The dipole coupling constant is not $\mu_0/4\pi$** — it's an arbitrary scale chosen for realistic nT amplitudes. The generator is "physics-inspired, not physically validated."
3. **The growth gate passes too easily.** The generator grows defects by exactly 15% with no noise, so a correct estimator recovers $\ln(1.15)$ almost exactly. It proves the arithmetic, not that the method works on real, irregular growth.
4. **Severity exceeds 100 %SMYS at run 2** (max 105.63). The contract says [0, 100]; no range check enforces it.
5. **coverage DQ check is inactive** (no caller passes `expected_length_m`).
6. **No cloud / S3 / real deploy.** Designed, costed, written down — not built, because no cloud account exists. Stated rather than faked.
7. **Growth-rate estimation uncertainty is not propagated** into remaining-life intervals — only the severity conformal interval is. Deliberate, stated scope limit.
8. **Consequence weights are engineering judgment**, not data (SCC 1.0, corrosion 0.6, dent 0.5, weld 0.4, interference 0.0). Meant to be replaced wholesale when real data exists.

16. Vocabulary — words that are easy to mix up

Word	Field	Means	Here
interference	physics	unwanted signal	buried junk metal beside the pipe
inference	ML	running a trained model on new data	<code>lsm predict <survey_id></code>
dug / dig	plain English	to make a hole in the ground	an inspected location; <code>dug indication</code> = a hole we made
%SMYS	pipeline engineering	percent of Specified Minimum Yield Strength	the severity unit; 100 = limit state
indication	NDT	a candidate finding, not yet confirmed	one clustered run of above-threshold rows
chainage	surveying	distance along the pipe from a datum	<code>chainage_m</code> , derived as <code>sample_idx × 0.5</code>
standoff	LSM	sensor-to-target distance	1.5 m here
challenger / champion	MLOps	candidate vs live model	everything is challenger; nothing promoted

17. Feature glossary — what `rx`, `gx`, etc. actually are

Companion to Section 6's table. Verified against `src/lsm/features.py`, not paraphrased from memory.

Residual block (background-removed signal):

Column	Formula	What it means
<code>rx_nt</code> , <code>ry_nt</code> , <code>rz_nt</code>	detrended residual, per axis (2-stage detrend, Section 6)	What's left in each of the 3 magnetometer axes after subtracting the ~45,000 nT background. This is the "signal," not the raw reading.
<code>r_mag_nt</code>	$\text{sqrt}(rx^2 + ry^2 + rz^2)$	Combined residual strength — one number regardless of orientation. Feeds almost everything downstream.
<code>r_incl_deg</code>	$\text{arcsin}(rz / r_mag)$, degrees	Inclination — how much the residual vector points "up/down" vs. sideways. Near a real defect this stays roughly constant as you pass over it (one solid buried object); near noise it's random sample-to-sample.
<code>r_decl_deg</code>	$\text{arctan2}(ry, rx)$, degrees	Declination — the compass direction the residual points in the horizontal plane. Same "stable near a defect, random on noise" logic as inclination.

Along-track gradient (one moving sensor — *not* gradiometry):

Column	Formula	What it means
<code>dr_ds_nt_per_m</code>	$d(r_mag)/ds$ (<code>np.gradient</code>)	Slope of the residual magnitude as you walk forward — how fast the signal changes per meter.
<code>d2r_ds2_nt_per_m2</code>	$d(dr_ds)/ds$	Curvature — the second derivative. Picks up sharp bends a first derivative misses.
<code>drz_ds_nt_per_m</code>	$d(rz)/ds$	Slope of just the vertical-axis residual (signed, not magnitude).

Vertical gradiometry (two stacked heads, only if the survey has a second head):

Column	Formula	What it means
<code>gx_nt_per_m</code> , <code>gy_nt_per_m</code> , <code>gz_nt_per_m</code>	$(\text{top_axis} - \text{bottom_axis}) / \text{baseline_m}$, per axis	Genuine two-sensor difference divided by their 0.5 m separation. Both heads see nearly the same distant background, so subtracting cancels it — true common-mode rejection (unlike the along-track gradient above).
<code>g_mag_nt_per_m</code>	$\text{sqrt}(gx^2 + gy^2 + gz^2)$	Combined gradiometer strength.

Section 7's negative result: doesn't actually help here — detrend already removed most of the background, so the two-sensor subtraction mostly adds noise. Kept in the feature set anyway.

Sliding-window stats (28 columns: 7 stats × 4 window sizes — `w2m` , `w5m` , `w10m` , `w25m` , all rolled over `r_mag` except `zcr`):

Suffix	Formula	What it means
<code>_mean_nt</code>	rolling mean	Average signal level in the window.
<code>_std_nt</code>	rolling std	How much it wobbles in the window.
<code>_max_nt</code>	rolling max	Tallest point in the window.
<code>_ptp_nt</code>	rolling max – rolling min	Peak-to-peak range.
<code>_kurt</code>	rolling kurtosis	How "spiky" vs. flat the distribution in the window is (needs ≥4 points; NaN below that).
<code>_zcr</code>	rolling mean of sign-flips on <code>rz</code> , not <code>r_mag</code>	Zero-crossing rate — how often the <i>signed</i> vertical residual flips sign. Computed on <code>rz</code> specifically because <code>r_mag</code> is always ≥ 0 and can never cross zero (a <code>\ r\ </code> -based ZCR would silently always be 0).
<code>_energy_nt2</code>	rolling sum of <code>r_mag²</code>	Total signal energy in the window.

Example: `w25m_kurt` = rolling kurtosis of `r_mag` over a 25 m window — the single highest defect-vs-interference separability column in the project (PR-AUC 0.985, Section 4).

Peak shape (5 columns, computed once per detected peak, then broadcast to nearby rows):

Column	What it means
<code>fwhm_m</code>	Full Width at Half Maximum — how wide the peak is at half its height. Narrow (~2.2 m) = defect-like; wide (~6.2 m) = interference-like. The single most important shape cue in the project.
<code>peak_asymmetry</code>	<code>((right_half_width) - (left_half_width)) / total_width . +1</code> = slow trailing flank (skewed right), <code>-1</code> = skewed left, <code>0</code> = symmetric.
<code>decay_exponent</code>	How steeply the peak's flanks fall off. <i>Didn't actually work</i> — <code>-0.95</code> on-pipe vs. <code>-0.84</code> off-pipe, too close to separate. Kept, with that negative result documented (Section 6).
<code>peak_prominence_nt</code>	How tall the peak stands above its <i>local</i> surroundings (not just its absolute height) — <code>scipy.signal.find_peaks</code> 's prominence.
<code>peak_distance_m</code>	Distance from this row to the nearest detected peak.

Rows with no peak nearby get `NaN` on all five (a real "nothing here" statement, not imputed to 0 — LightGBM splits on missingness natively).

18. Question & answer — plain-language session notes

This section writes out, in plain and detailed everyday language, the questions that came up in conversation while studying this project, together with their answers. The rest of this document is written in a dense, shorthand style meant for fast recall once you already understand the ideas. This section is the opposite on purpose: longer, slower, and written so that someone who has never seen the code before could follow it just by reading straight through.

Question: When we use a rolling window to remove the background by subtracting the median, doesn't that also remove the outliers, since the median is the center of the data and outliers are far from the center?

This is a very reasonable thing to worry about, but the way it actually works avoids the problem, and it helps to walk through it slowly.

We are trying to find the "normal, everyday" background level inside a 40-meter-wide window of readings, so that we can subtract that background and see only what is left over — which, if there is a real defect nearby, should be the defect's own signal. To do this, at every position along the pipe, we look at the 40 meters of readings surrounding that position, and we compute one single number from that window: the median of all the readings inside it. The median just means: sort all the values in the window from smallest to largest, and take the one exactly in the middle.

The important thing to notice is what this median number is actually used for. It is not used to change or "clean" any of the individual raw readings. It is only used to estimate one number — the background level at that position. Once we have that one number, we go back to the original raw readings, which still contain the full, untouched defect signal exactly as it was measured, and we subtract the background number from them. So the raw defect signal is never touched or trimmed by the median calculation itself — only the number we choose to subtract from it is computed using the median.

Now, why does using the median instead of a plain average work so well here? A real defect is physically narrow, roughly 2 meters wide, compared to the 40-meter window we are using to estimate the background. That means that inside any one window, the handful of readings that are affected by a defect make up only a small fraction of all the readings in that window, something like 5 percent. When you sort all the readings in the window and pick the middle one, that middle value will almost always land among the 95 percent of "ordinary" readings, not among the small number of defect-affected ones. In other words, the median naturally represents the background level and stays basically blind to a small minority of unusual values sitting to one side.

If we used a plain average instead, this would not be true anymore. An average is influenced by every single value in the window, including the defect-affected ones. If there is a real bump in the window, the average would be pulled a little bit toward that bump. Then, when you subtract this average from the raw readings, you would be subtracting away part of the defect's own signal along with the background, because part of the defect had already leaked into what you called "the background." This would make the defect look smaller than

it really is in the final result. This is exactly why an average is not used here — it partially erases the very thing we are trying to detect, while the median does not.

So, to summarize plainly: the median does not throw away outliers from the data itself — the raw data is never altered. The median is only used to get an accurate reading of the background level, and because it ignores a small number of unusual values when computing that one number, the background estimate stays accurate even in a window that happens to contain a real defect.

There is a real limit to this, though, and it matches the intuition behind the original question. This only works because the defect only ever makes up a small minority of the window (about 2 meters out of 40 meters). If the window were much narrower, or if defects were packed close enough together that they started to dominate a window, the defect-affected points could become the majority of the window, and then the median itself would start tracking the defect instead of the background — and subtracting it really would erase the defect's own signal. This is exactly why the project is careful to keep the window much wider than a defect's own physical size.

Question: Can we then define a confidence interval for the median?

Yes, a median can absolutely have a confidence interval, even though it is a little less automatic than it is for a plain average.

A confidence interval is a way of expressing how much you should trust a single number you calculated from a limited sample of data. Instead of saying "the median is exactly 5.2," you say something like "the median is probably somewhere between 4.8 and 5.6, and we are 95 percent confident about that range." This tells you how much that number might shift if you had collected a slightly different sample of similar data.

There are three common ways to build this kind of interval for a median.

The first way makes no assumption at all about the shape of the underlying data. You sort your data points from smallest to largest. Using only basic probability (specifically, the fact that each point independently has a 50/50 chance of falling above or below the true median), you can calculate exactly which two of these sorted values will contain the true median 95 percent of the time. This method is cheap to compute, since it does not require any simulation, and it does not depend on assuming the data follows any particular shape, like a bell curve.

The second way assumes the data roughly follows a bell-curve shape (a "normal distribution"). In that specific case, there is a known relationship: the typical uncertainty of a median is about 1.25 times bigger than the typical uncertainty of an average computed from the same data. This is a convenient shortcut, but it is only trustworthy if the bell-curve assumption actually holds, which is not guaranteed for a 40-meter window that might contain a real bump.

The third way is called bootstrapping, and it is the method already used everywhere else in this project for other measurements. The idea is: take your set of data points, and randomly pick that same number of points again, but this time with replacement, meaning the same point can be picked more than once and some points might not be picked at all. Compute the median of this new randomly resampled set. Now repeat this whole process

many times, for example a thousand times, each time getting a slightly different median because each resample is slightly different. After doing this a thousand times, you have a thousand different median values, and the middle 95 percent of them gives you your confidence interval. This method does not assume anything about the shape of the data, which makes it the most flexible of the three, and it matches the technique already used throughout this project to build confidence intervals for other measurements, like recall or coverage.

There is one important warning specific to this situation. All three of these methods assume that the data points being used are independent of each other. But readings right next to each other in a rolling window along a magnetometer survey are not independent — they are correlated, because the background field changes slowly and smoothly as you walk along the pipe. If you pretend these correlated points are independent, the confidence interval you compute will come out artificially narrow, meaning it will look more confident than it really should be, because you are behaving as if you have more genuinely independent information than you actually do. The proper fix is something called a block bootstrap, where instead of resampling individual points one at a time, you resample small contiguous chunks of the data together, which preserves the natural correlation between neighboring points and gives an honest interval instead of a falsely narrow one.

Question: Can you create a table explaining what `rx` , `gx` , and the other feature columns actually are?

The full answer to this question is written out as its own dedicated section, Section 17 above ("Feature glossary"), rather than repeated here, since it is naturally a table of individual columns rather than a flowing explanation. In short: `rx` , `ry` , `rz` are the sensor's three individual measurement directions after the background has been removed, `r_mag` is their combined overall strength, `r_inc1` and `r_dec1` describe which direction the disturbance points in, the `gx` / `gy` / `gz` columns come from comparing two physically separate sensor heads stacked on top of each other, and the remaining columns are various statistics computed over small sliding windows of data, plus a handful of columns describing the shape of any detected bump. Section 17 lists every column individually with its exact formula and plain-language meaning.

Question: I don't fully understand what the ML pipeline is doing when it talks about "cheating" — the part about the function only being handed one survey's data.

"Cheating" here refers to something in machine learning usually called data leakage. It means that while a model is being trained or tested, it accidentally gets access to some piece of information that it would not actually have available once it is genuinely being used out in the real world. Because the model had access to that extra information during testing, it will score very well on the test, but that good score is misleading — it does not reflect how the model would really perform once deployed, because in the real deployed system that extra information is simply not available at the moment a prediction needs to be made.

Here is a simple, completely generic example, unrelated to this project, to make the idea clear. Imagine you are building a model to predict house prices. Before splitting your data into a training set and a separate test set, you calculate the average and the typical spread of all the house prices across your entire dataset, and you use those numbers to rescale your input features. The problem is that this "entire dataset" includes the test set. So a small amount of information about the test set's own prices has quietly influenced how the training data was

prepared. When you later evaluate the model on that same test set, it will look unfairly good, because the test set subtly helped shape the training process to begin with. But when a genuinely new house comes along in the real world, you obviously cannot peek at its price ahead of time to help process it. So the model will perform noticeably worse on real new houses than the test score had promised.

Now, closer to how this exact kind of mistake could happen in this specific project: imagine a feature defined as "how unusual is this defect's reading, compared to the average reading across every survey in the whole database." If that average were computed using surveys that were recorded chronologically after the survey currently being scored, the model would quietly be using knowledge of the future to help judge the present. In a backtest done after the fact, this looks completely fine, because all the data already exists by the time you run the calculation. But in real production, at the moment a brand-new survey comes in and needs to be scored immediately, those later surveys genuinely do not exist yet. So this would be an unrealistic advantage during testing that disappears the moment the system is actually used for real.

So how does this project prevent this specific kind of mistake? Every function in the code that computes a feature for one survey is written so that the only information it is ever handed is the raw sensor readings from that one single survey, plus a small handful of basic facts describing that survey — things like its identifying name, which physical pipeline it belongs to, which visit number it is, the spacing between samples, the sensor's height above the pipe, when it was surveyed, and the distance between the two sensor heads if a second head is present. The function is never given, and has no way to obtain, a connection to the database or any reference to any other survey, let alone a statistic computed across the whole collection of surveys.

Why does this matter more than simply writing a note in the code saying "please do not use information from other surveys"? Because a note like that is only a suggestion. A developer could still, by accident or convenience, write code elsewhere that pulls in a database connection and computes some statistic across everything, and nothing in the programming language itself would stop them from plugging that into a feature. But by designing the function so that it can only ever be handed a single survey's data and nothing else, this kind of mistake becomes genuinely impossible to write, because there is simply nothing inside the function that refers to any other survey in the first place. The safety here does not depend on a developer remembering a rule — it comes from what data the function is even allowed to receive.

Question: Why do we merge the three x, y, z sensor readings into one overall magnetic response? Doesn't that lose the orientation, like whether a defect is on the left or right side of a 50 cm diameter pipe?

There are actually two different things being asked here, and it helps to separate them, because the original question quietly combines them into one.

The first thing is the orientation of the magnetic disturbance itself, meaning: which direction does the anomaly point in, at the moment the sensor passes over it. Every magnetized object, such as a corroded patch of steel, has its own internal magnetic direction, somewhat like how a small bar magnet has a north end and a south end pointing some particular way. This direction is a property of the object itself, and this project does keep track of it — there are two dedicated columns, called inclination and declination, that record exactly which direction the

measured disturbance points in, at every point along the survey. So this kind of orientation information is not thrown away.

The second thing, which is genuinely different, is the physical position of the defect relative to the pipe — specifically, whether it sits on the left side, the right side, the top, or the bottom, going around the pipe's round cross-section. This is sometimes described using a clock face, like saying a defect sits at the "3 o'clock position" around the pipe. This is a question about where the defect physically is, not which way its magnetism points.

Earlier, the reason given for combining the three individual sensor-axis readings into one overall strength number was specifically about finding and measuring the size of anomalies along the survey — that step needs one single number per location so that a peak-finding calculation can look for bumps in it. That process is not attempting to answer the "which side of the pipe" question at all, and combining the three axes into one number is not what causes that information to be lost.

The real, more fundamental reason this project cannot currently answer "which side of the pipe is the defect on" has nothing to do with combining the axes. It comes from how the training data itself was generated in the first place. When the code generates a synthetic defect for training, it always places that defect exactly on the sensor's own path, with zero sideways offset — always centered directly under where the sensor walks. Only the other category of object, called interference (things like a nearby buried fence post or scrap of metal, which are not real pipeline defects at all, just clutter sitting beside the pipe), is given a random sideways offset in the training data. Because of this, the training data never contains any example of a real defect sitting at different positions around the pipe's circumference — the model was never shown any variation of that kind, so there is nothing for it to have learned about telling left from right for a real defect, no matter which features it was given.

To actually build a real tool that could answer "which side of the pipe is this on," you would need either several magnetic sensors mounted at different positions around the pipe's circumference at the same time, or a more advanced mathematical approach that works backward from the full three-direction magnetic signal to solve for exactly where in three-dimensional space the source object must be. This project does neither of those things. It only tries to answer "is there likely a defect here, and how severe does it look," not "exactly where around the pipe's circumference is it located."

Question: Do we use cylindrical space to model the pipe, or something else?

No, this project does not use cylindrical coordinates, and it does not model the physical shape of the pipe at all.

The way the program generates a defect's magnetic signal is very simple. It picks a single point in ordinary, flat, three-dimensional space, described using an along-pipe position, a sideways position, and a depth, and it treats that point as if it were a tiny magnet sitting there. It then uses a standard physics formula to calculate how strong the resulting magnetic field would be at the sensor's position, based on the straight-line distance between that point and the sensor. Nowhere in this calculation is the pipe represented as an actual round tube with a wall and a circular cross-section. The pipe's physical shape is never really modeled at all — it is only implied by where we happen to place these single-point magnetic sources. So there is no cylindrical coordinate

system involved here, just an ordinary flat 3D coordinate system with a point-shaped magnetic source placed somewhere near where the pipe is assumed to be.

Question: But in reality, a dipole moment doesn't fall off as one-over-distance-cubed, does it?

There is a small but important distinction being mixed up here, and it is worth separating clearly.

A magnetic dipole moment is a fixed number describing how strong a magnetic source is — similar to describing the overall strength of a small bar magnet. This number does not get weaker just because you move farther away from the object; it is a fixed property of the object itself and does not depend on distance at all.

What does get weaker with distance is the actual magnetic field that this fixed-strength source produces at some other point away from it — in other words, how strong of a magnetic effect a sensor placed some distance away would actually measure. This measured field strength genuinely does fall off very quickly as you move farther away, specifically in proportion to one divided by the distance cubed. This is not something made up for this project — it is standard, well established physics, exactly true for an idealized point-shaped magnetic source, meaning a source small enough that its own physical size does not matter.

The more interesting and legitimate criticism is this: a real corrosion patch, crack, or dent is not actually an infinitely small point. It has some real physical size, possibly tens of centimeters across. The one-over-distance-cubed formula is only exactly correct if you are looking at a truly point-sized source, or if you are far enough away that the source's own size stops mattering, which is usually called being in the "far field." Since the sensor typically flies only about a meter and a half above the pipe, and a real defect might be a noticeable fraction of that distance across, it is not guaranteed that we are far enough away for the simple point-source formula to perfectly describe the real physics. There could be extra, more complicated field patterns up close that this simplified model does not capture.

There is also a bigger, more fundamental difference worth knowing about. Real pipeline inspection tools that use magnetism often do not work by measuring some separate small magnet buried near the pipe at all. Instead, they magnetize the steel pipe itself, and then look for places where a defect, like wall thinning from corrosion, causes the magnetic field to "leak" out of the pipe wall in an unusual way. This is a completely different physical mechanism from "a small magnet floating near the pipe, producing a field out in open space," which is what this project's generator actually simulates. So this generator should be understood as a simplified, physics-inspired stand-in for the real phenomenon, not an accurate physical simulation of how real pipeline inspection tools genuinely work.

Question: Can you explain more about what split conformal calibration is, and whether the held-out data comes before the train and test split?

The severity model does not just predict a single number for how severe a defect is — it predicts a range, for example "probably somewhere between 40 percent and 60 percent severity, with 50 percent being the most likely single value." This is more useful to an engineer than a single number, because it also tells them how confident to be. Producing this range happens in two separate steps.

The first step is a machine learning model, specifically one built from many small decision trees, that is trained to directly predict three specific values for each defect: the value below which only 5 percent of real severities would be expected to fall, the typical or middle value (the 50th percentile), and the value above which only 5 percent of real severities would be expected to fall. Together, the 5 percent value and the 95 percent value form a range that is supposed to contain the true severity 90 percent of the time.

The problem is that a model like this, straight out of training, usually cannot be fully trusted to actually deliver that promised 90 percent. When this project actually measured it directly, the raw predicted range only contained the true severity about 73 percent of the time, not the promised 90 percent. This is a well known issue with machine learning models that try to directly predict their own uncertainty — they tend to be overconfident, meaning their predicted ranges usually come out too narrow.

The second step, called conformal calibration, is the fix for this. The idea is to take a separate batch of data that the model was not trained on, and measure, using real examples the model has never seen, exactly how wrong its predicted ranges actually are. For every example in this separate batch, you check whether the true severity fell outside the model's predicted range, and if so, by how much. Doing this for every example in the batch gives you a whole list of numbers describing how far off the ranges typically were. You then take a specific position within that list, roughly comparable to a high percentile of those numbers, and use it as a single fixed correction — you stretch every future prediction's range outward by that same amount, on both the low end and the high end. Because this correction was measured using data the model genuinely never touched during training, it tends to actually fix the coverage problem, bringing the real "how often is the true answer inside my range" number back up close to the promised 90 percent, even for brand-new data seen later.

Now, to directly answer when and how this held-out data is separated from everything else: this project actually uses three separate groups of data, not the usual two of just training and testing.

First, the data is divided into five roughly equal groups, based on which physical stretch of pipeline each row comes from, making sure that data from the same physical stretch never gets split across two different groups, since that could make the evaluation look artificially better than it really is. One of these five groups is set completely aside as the final test set for that round. This data is not touched again until the very end, when we want to check how good our predicted ranges truly are.

The other four groups combined form what can be called the working data. This working data is then split again into two smaller pieces. Most of it becomes the actual training data used to fit the machine learning model that predicts the 5 percent, 50 percent, and 95 percent values. A smaller portion of it, a specific fraction set aside separately, becomes the calibration data used only for the second step above, measuring how wrong the model's ranges are in order to compute the fixed correction amount.

So the calibration data is not the same thing as the final test data. The calibration data is used only to figure out how much to widen the prediction ranges. The final test data, kept completely separate the entire time, is used only at the very end, to check and report how good the corrected ranges actually are on data that had no role whatsoever in either training the model or calibrating its ranges. This careful three-way separation matters, because if you calibrated and then reported your results using the exact same data, your final "it works 90

percent of the time" number would be dishonestly optimistic, in the same sense as the "cheating" discussed earlier in this document.

One more detail worth explaining plainly: when splitting the working data into the training piece and the calibration piece, this project is careful to do that split based on which distinct physical defect each row belongs to, rather than splitting individual rows randomly. This matters because the same physical defect can be surveyed more than once, for example as a follow-up inspection visit, which produces multiple rows of data describing that same underlying defect. If rows were split randomly, some rows from one visit to a defect could end up in training while other rows from a different visit to that exact same defect end up in calibration. Since the model could then partly "recognize" that specific defect from having seen it during training, this would leak a small amount of information between training and calibration, in the same "cheating" sense discussed earlier. By splitting according to which distinct defect a row belongs to, the model never partly learns about a specific defect that it will later be calibrated or tested against.

Question: So the conformal calibration step uses the trained model to predict on the held-out data and checks the low and high predictions against the true value?

That is very close to correct, with one small but important refinement: it is not a simple yes-or-no check of whether the true value landed inside the predicted range. Instead, for every single example in the calibration data, the exact size of the miss (or the exact size of the safety margin, if there was no miss) gets measured and recorded as a number.

Concretely, for each calibration example, the model's predicted low value and predicted high value are compared against the actual true value. If the true value falls outside the predicted range, above the high value or below the low value, the size of that overshoot is recorded as a positive number. If the true value falls comfortably inside the predicted range, the amount of extra room to spare is recorded as a negative number. Doing this for every calibration example produces a whole list of these measurements, one number per example.

Once this whole list exists, a specific position within it, roughly a high percentile of the list, is picked out and used as the single correction amount that gets applied to every future prediction, stretching its predicted range outward by that same amount on both the low and high sides. So it is not literally "catching" individual predictions in a pass-or-fail sense — it is measuring, precisely, by how much the model's raw predictions missed or didn't miss on data it never trained on, collecting all of these measurements together, and using a summary of that whole collection as the single correction applied afterward.

Question: Do we shuffle the data before splitting it?

Yes, in one of the two places data gets divided into groups we do shuffle it, but not in the other place, and there is a specific reason for the difference.

When dividing the working data into the training piece and the calibration piece, described in the previous answer, the project does randomly shuffle the list of distinct physical defects before cutting off a portion of that shuffled list to become the calibration group. This shuffling uses a fixed starting point, usually called a seed,

which means that even though it looks random, running the exact same code again will always produce exactly the same shuffled order and exactly the same split. So it is reproducible, not truly unpredictable, even though it behaves like a genuine random shuffle. Shuffling is needed here because, without it, simply taking "the first few defects in the list" would risk taking defects that happen to share some property, such as being located near the start of the pipeline, which could unfairly bias which defects end up in calibration versus training.

For the other split, the one that divides the entire dataset into the five main groups used for cross-validation, the project deliberately does not use an actual shuffle. Instead, it uses a technique called hashing, which takes each pipeline segment's identifying label and converts it into a large number through a fixed, repeatable calculation, and then uses the remainder of that number when divided by five to decide which of the five groups that segment belongs to. This achieves basically the same practical effect as a shuffle would, spreading segments out across the five groups in a way that doesn't depend on their original order in the file, but it has an extra advantage that a true shuffle would not have. If new pipeline segments are later added to the dataset, for example from a newly recorded survey, this hashing method guarantees that every previously existing segment stays in exactly the same group it was already assigned to. A genuine shuffle-then-split approach does not have this property — adding new data and reshuffling could completely change where all the old data lands too, making it impossible to fairly compare results from before and after adding new data.

Question: Are you using K-fold cross-validation in general, and is it something like 10 folds?

The project does use cross-validation, which is a way of testing a model fairly using all of the available data, by repeating the train-and-test process multiple times, each time holding out a different slice of the data as the test set and using the rest as training. The number of times this is repeated, and therefore how many roughly equal slices the data gets divided into, is usually called K. In this project, K is 5, not 10.

It also matters *how* each row gets assigned to one of those 5 groups, not just how many groups there are. This project never groups rows purely by physical pipeline stretch, in the order the data happens to appear. Instead, every row is first tagged with a label describing which 100-meter stretch of which physical pipeline it came from. That label is then run through a hashing calculation — a fixed, repeatable mathematical formula that turns any piece of text into a large number — and the remainder left over when that large number is divided by 5 decides which of the 5 groups the row goes into. This is explained in more depth in the "do we shuffle the data" answer above; the short version is that this hashing approach spreads rows across the 5 groups in a way that looks essentially random, without actually depending on a random shuffle, and — importantly — it guarantees that every row from the same 100-meter stretch of the same pipeline always lands in the same group, so a model is never accidentally trained on part of a stretch and tested on a different part of that exact same stretch.

The reason 5 was chosen instead of a larger number like 10 comes down to how little real data exists in the demo-sized version of this project — there are only around 12 real physical defects in total. If the data is split into 5 groups, each held-out test group ends up containing only about 2 or 3 of those defects, which is already a very small number to compute a meaningful statistic from. If 10 folds were used instead, each held-out test group would shrink to roughly just 1 defect, which would be far too few to calculate anything statistically meaningful from at all. So 5 was chosen as close to the smallest reasonable number of folds given how few real

defects exist in the demo-sized dataset, rather than being picked simply because "10 folds" is a common default elsewhere.

There is a second, related use of this same "5 folds" idea in the larger-scale version of this project (referred to elsewhere as the scale rehearsal, which simulates a much bigger, more realistic dataset instead of the small demo one). In that larger version, instead of splitting the data into small 100-meter chunks within pipeline lines, the project holds out entire, complete physical pipeline lines at a time, specifically to test whether the model still works on a totally different, never-before-seen stretch of pipeline, rather than just a different small chunk of a pipeline it has already partly seen elsewhere. Even in that larger-scale version, 5 was kept as the number of folds, rather than using a separate fold for each of the 40 individual pipeline lines in that larger dataset, because 5 folds still gives each held-out group around 8 full lines' worth of real defects to test against, which is already a solid, statistically meaningful amount. Going all the way up to 40 individual folds would mean each fold only tests against 1 line at a time, providing very little additional statistical benefit for a lot of extra computation.